CSC2620 - Object Oriented Design Final

S.H.A.C. Waffle Games

By Lucas Gasper, Erik Fudge, & Jack Griffin

Instructor: Dr. Jhansi Rani Prathuri

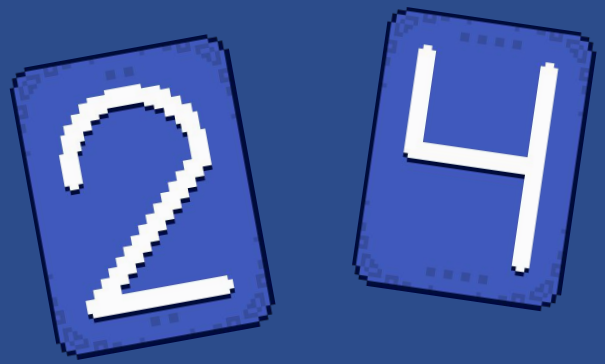
May 5th, 2026

Introduction

Our project is a small collection of multiplayer games that allows users to join and play against one another from separate devices connected on the same network. *S.H.A.C. Waffle Games* solves the problem of trying to play games online with others by keeping it a local host, all while staying interesting through its incorporation of advanced Java features, including networking, GUI, and object-oriented architecture.

Problem Statement & Domain Application

The domain is our networked multiplayer games all over LAN, with our users being a small group of 2-4 players, all on their own computers. The main challenges addressed are establishing a real-time, peer-to-peer system for communication through a host/client model, synchronizing the game state between players, and having clean, readable interfaces for the users.



System Overview

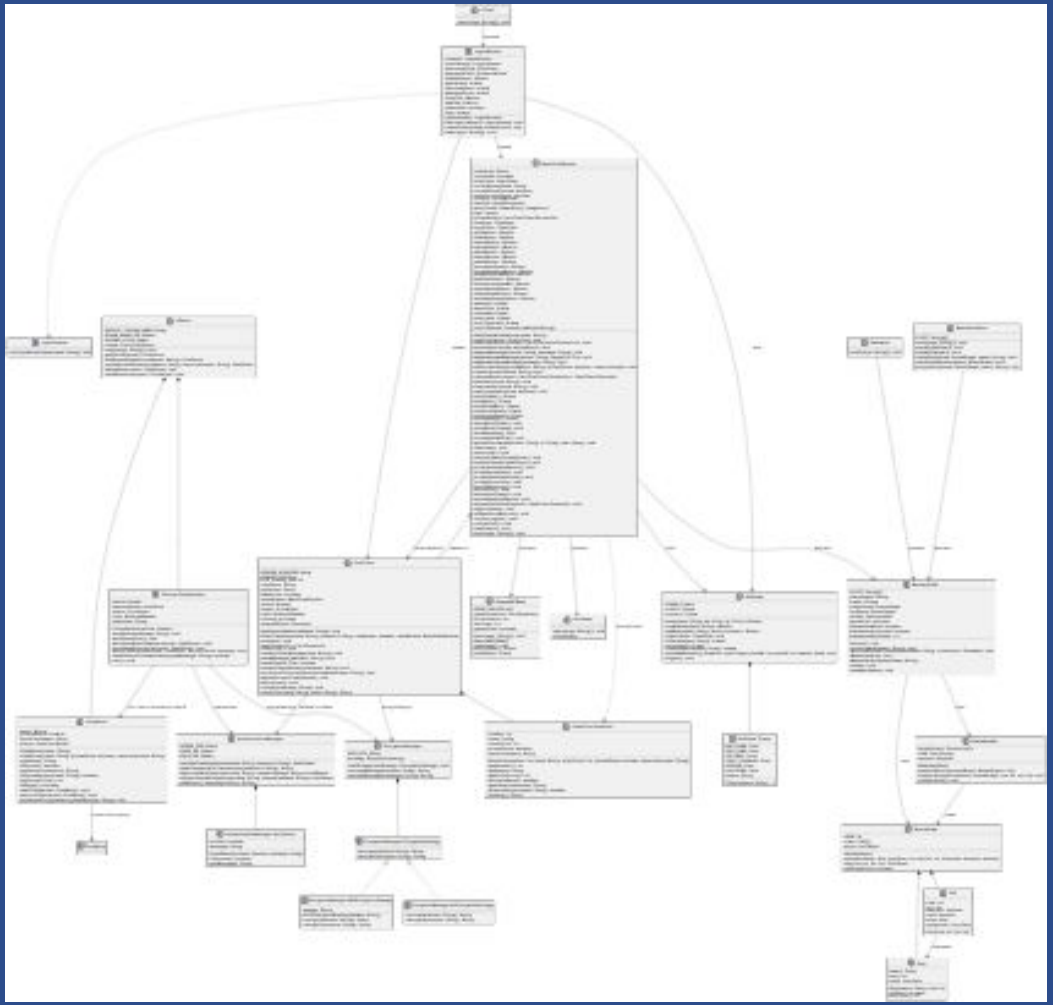
As previously mentioned, *S.H.A.C. Waffle Games* is a LAN-based game collection program, that allows players to login and connect with other users. One computer is designated as the host, and other users enter their IP address before running the application. While all the games feature live synchronization and chat systems with export functionality, the most prominent of all these games, *SHACUNO*, has even more features, including custom-made cards and color tinting. Additionally, for the sake of time, *SHACUNO* will be the focus of the slides.

OOAD Approach

The analysis process started with the core idea of the card game domain: cards, decks, players, and the game rules. From there, classes were made to handle the responsibilities: the Deck manages drawing and discarding, Card encapsulates type and color data for each card, PlayerInfo tracks the state of hands, and UnoGame itself serves as the central controller and view of the program.

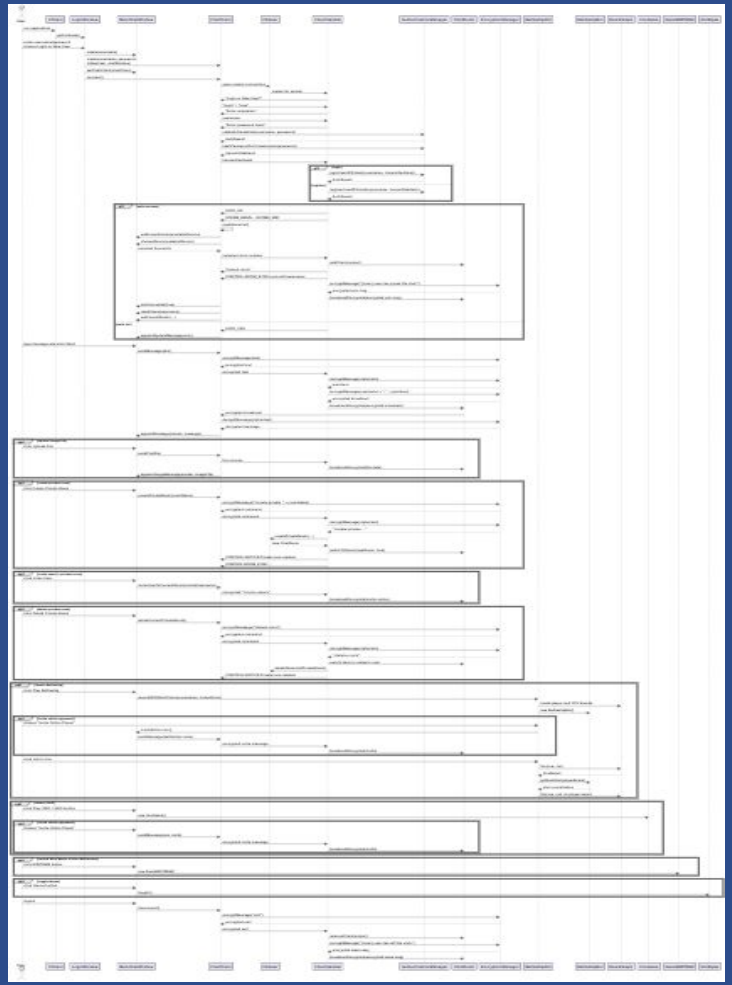
Class Diagram

- Added new classes for each game
- New buttons added
- Added relationship arrows to all the new classes



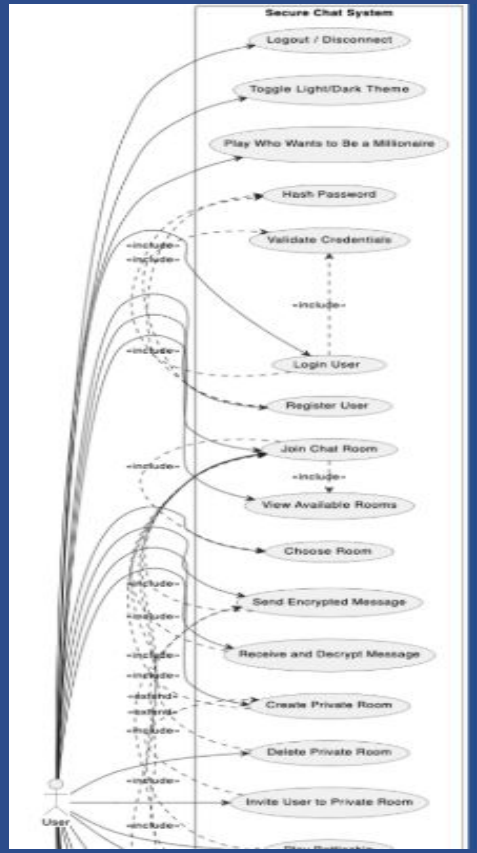
Sequence Diagram

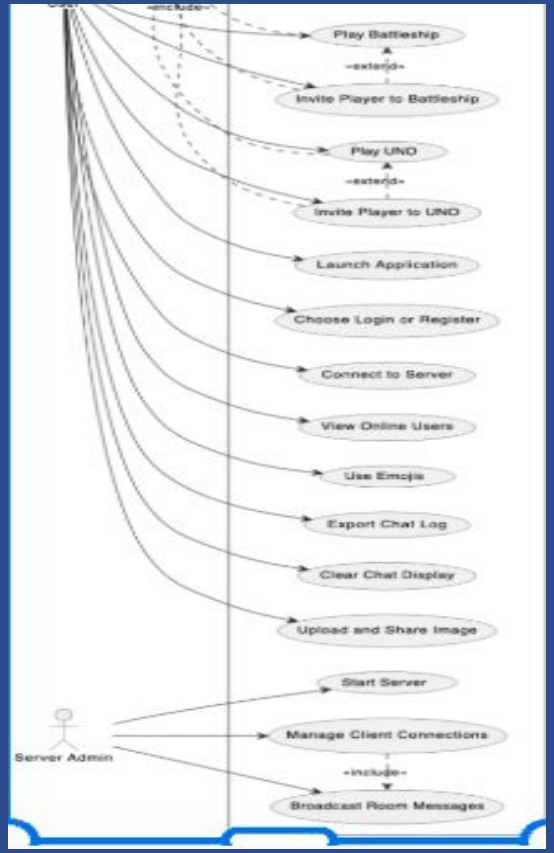
- Added new classes for each new game as participants
- Added new blocks showing each game is launched from the MainChatWindow



Use Case Diagram

- Added new use cases for each game
- Added use cases for game invites





Design Patterns Used

Observer - Implicit via Swing event listeners	Card click handlers and button listeners observe the UI events and trigger the respective game logic, and is used throughout the hand panel and draw pile.
Singleton - Deck	Only one Deck instance exists per game session, which is created at the start of the game and referenced by the host for the all draw/discard operations.
Strategy - Bot Logic (solo testing)	For testing purposes, the program can have two bots take the place of other players. Their decision-making within botTakeTurn() is isolated and could be swapped for better AI without changing other code.
Factory - Card Creation	The Deck constructor acts as a factory, as it generates a full set of 108 cards, rather than constructing cards at the call site.
State Pattern - Game Flow	The gameStarted boolean and currentTurn index together represent the games state, controlling which actions can be taken at any moment.

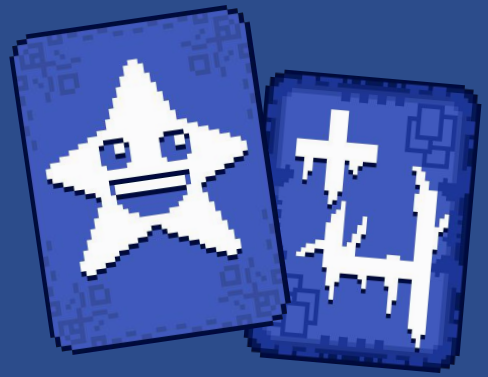
Multithreading Component

Threads are safely maintained by routing all UI updates through SwingUtilities.invokeLater(), so components are only modified on the Event Dispatch Thread.

Server accept loop	A background thread in startHost() continuously listens for incoming client connections, which prevents the UI from freezing while waiting.
Per-client listener threads	Each connected client gets their own thread running listenToClient(), which reads incoming NetMessages.
Client-to-host listener	joinServer() spawns a thread running listenToHost() to receive state broadcasts from the host.
Bot delay timer (solo testing)	javax.swing.Timer fires bot actions after a 1.2 second delay, to prevent bots from instantly taking their turn.
Rainbow animation	Another timer continuously updates the hue shifting every 50ms for the wild and +4 cards.

Java Swing GUI

The GUI of *SHACUNO, SHACBATTLESHIP, and SHACWWTBAM* includes multiple JFrames, JTextAreas, JPanels, JLabels, and JButtons, all serving different purposes. Key functionalities include playable cards gaining a yellow highlight around them, an info box at the top displaying information on who's turn it is and what the top card is, and layout managers ensure the UI scales with window resizing.



Regular Expressions

Regex is used in the networking setup to validate the IP address input before attempting a socket connection, which prevents malformed addresses from causing unhandled exceptions.

Regex improves reliability by providing immediate and clear feedback to the user rather than letting the connection attempt fail or throw an exception.

Socket Programming

The architecture comprises of a Host-authoritative model, as one player hosts and others connect via a TCP socket using the host's IP.

Communication flow: Client connects and sends player name as object. Host adds player, acknowledges via broadcast. Host starts the game, serializes full GameState and sends to all clients. Clients send NetMessage objects, and finally host validates, updates state, broadcasts new GameState to all clients.

Data is exchanged through serialized Java objects, including GameState, NetMessage, and String. Each client is handled on its own dedicated thread, allowing the host to listen to all players simultaneously.

System Architecture

SHACUNO uses the GUI layer to render visuals and capture user input, the Game Logic layer validates correct moves, applies the rules, and mangoes turn order.

The Model layer handles pure data and domain logic, while the Networking layer serializes and transmits game state and player actions.

The Threading layer keeps the UI responsive while listening for inputs and outputs.

Live Demo

We will be demonstrating a game (or two, if it goes by very quickly) of *SHACUNO*, as well as some of the other games to demonstrate their features. We will also open up the puml files for a bit more readability.

Limitations & Known Issues

The main issues and limitations are how there are no persistent accounts or scoring across sessions, the login is only for the current session. Additionally, reconnecting is another issue, as if a player disconnects mid-game, the game session cannot recover gracefully.

Regarding *SHACUNO*, there are some visuals that could've been touched up on, like having the wild cards rainbow effect stop and the hue set to whatever color was chosen. For solo testing, the bot's are really not the smartest, as they play the first eligible card in their hand, and pick random wild card colors, they don't really have any strategy.

Future Enhancements

Some features that would really help the project come to life more would be sound effects and moving components. Furthermore, being able to spectate games or having a more robust lobby system, or even in-game rules, could help users who are unfamiliar with the games available.

Team Contributions

Lucas Gasper	Create the GUI for Who Wants To Be A Millionaire (WWTBAM), created the UNO code, and play tested WWTBAM.
Erik Fudge	Created the GUI for Battleship, created the code for Battleship, and play tested all 3 games.
Jack Griffin	Created the GUI, cards, visuals, and play tested for *SHACUNO.* Created the slides presentation and overall layout, organized directory and folders.

Lessons Learned

Being able to use multiply-blend tinting on images (the cards) in *SHACUNO* was a big help, as only 15 cards needed to be made rather than 57 cards. Communication was a bit lacking for undergoing such a big project, which made trying to tie together some components and make every part of the project work a bit tricky.

Conclusion

To wrap up our presentation, *S.H.A.C. Waffle Games* is our project that emulates a fully functional, real-time multiplayer game collection within Java. Our application utilizes a wide range of OOD design principles, from encapsulation to design patterns, and works alongside advanced Java features such as multithreading, Swing GUI development, and socket programming.

Questions & Answers

Thank you for your time!